AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_HWSW_engineering_job Version : 1	

HW/SW PLATTFORM SYSTEM FÖR TASKING IDENTIFIERING

Darwin Rengifo
2020-09-28

Sammanfattning:
Denna rapport beskriver en förstudie och planering för att bygga en plattform som innehålla accelerometer från DE10-Lite kortet, CPU, VGA, Sierra, timer och sjusegmentdisplay med några special funktioner. Qsys-verktyget för att integrerar hårdvaran och mjukvaran interface genom läsningen och skrivningen på Avalon bus och en VGA skärmen för att visa resultatet när läser koordinaterna från accelerometer. Denna rapport innehåller aktivitets- och tidplan, projektkostnad, kravspecifikations, projektarkitektur och beskrivning av utvecklade komponenter.

Innehåll

1 Inledning 4

2 Kravspecifikation från kund 5

3 Beskrivning av två gamla projekt. 8

3.1 Lars Bengtsson: Space Invaders 8

3.2 André Norberg: Rabbit vs Robots 2 8

4 Första arkitektur 8

4.1 Pll 9

4.2 System ID 9

4.3 OnChip_Ram 9

4.4 OnChip_flash 9

4.5 Sju_segment ip 9

4.6 Vga ip 9

4.7 Jtag uart 9

4.8 Sdram 9

4.9 Sierra ip 9

4.10 Accelerometer spi 9

4.11 Timer ip 9

5 PROJEKPLAN 10

6 Bootningen beskrivning. 11

7 Hårdvaran konstruktions beskrivning 12

7.1 Systemarkitektur 12

7.2 Delsystem 13

7.2.1 Toppnivån 13

7.2.1.1 Systemarkitektur 13

7.2.2 Sjusegmentdisplay IP - komponents 16

7.2.2.1 Systemarkitektur 16

7.2.3 Timer IP - komponent 17

7.2.3.1 Systemarkitektur 17

7.2.4 Vga IP - komponent 18

7.2.4.1 Systemarkitektur 18

7.2.5 Metastabilitet lösning 19

7.3 I/O pinnar 20

7.4 Frekvens, storlek, effektförbrukning uppskattning 20
7.4.1 Max frekvens för hela konstruktionen 20
7.4.2 Storlek för hela konstruktionen och del system 21
7.4.3 Effektförbrukning uppskattning 22
8 Mjukvaran konstruktions beskrivning 23
8.1 Minneskarta 23
8.2 Task 23
8.2.1 Idle 23
8.2.2 Accelerometer_Task 23
8.2.3 Sjusegment_Task 23
8.2.4 Timer_Task_1 24
8.2.5 Timer_Task_2 24
8.3 Storlek 24
8.4 Inställningar 24
9 Validering 25
10 Analys 25
11 Slutsatser 25

1 Inledning

Denna rapport beskriver hur man bygger en komplett plattform som innehålla flera IP-komponenter. Konstruktionen ska använda Sierra Real-Time Kernel(RTK-accelerator) för att jobba med multi-tasking och genom sjusegmentdisplay visar vilken task som är på exekvering.

2 Kravspecifikation från kund

Tabell 1. Kravspecifikation från kund

Krav	Beskrivning	Utfört
Förstudie/startrapport (lägg även med detta i slutrapporten)		
F1	Beskriv två HW/SW-projekt från tidigare år som helst har någon likhet med det som ska utföras i detta ingenjörsjobb.	Ja
F2	Beskriv vad som ska utföras.	Ja
F3	Gör en första arkitektur som visar alla HW/SW komponenter (kan senare modifieras). Se krav längre ner. Beskriv också de komponenter som ska utvecklas, återanvändas från tidigare uppgifter och som andra har utvecklat, då med tydliga referenser.	Ja
F4	Aktivitets- och tidplan för projektet. Följ upp tidplanen och redovisa resultatet i slutrapporten. Lön är 800 SEK/tim.	Ja
Konstruktions krav		
F5	Inbyggt datorsystem på DE10-Lite med minst följande komponent krav: - en accelerometer, - VGA (egenutvecklad), - Sierra, - externt RAM minne, - timer (egenutvecklad) och - en eller flera special funktioner.	Ja
F6	Specialfunktionen är följande: • En 7 segmentsdisplayer visar vilken task som är i tillståndet ”RUN”. (enbart en HW lösning).	Ja
F7	Systemet ska levereras med ett demonstrationsprogram som är kopplad till en givare, t.ex. accelerometern. Det ska visas på VGA- skärmen med följande minimikrav: - Första sidan ska visa namn och systemet ska beskrivas (bild eller text)	Ja
	- Någon typ av loggning eller något liknande som bevisar programmets funktion	Ja
F8	All kod ska få plats på det interna RAM-minnet.	Ja
F9	Kortet ska boota upp både HW/SW automatiskt efter spänningspåslag.	

F10	Startrapporten ska innehålla: Framsida Sammanfattning Krav F1 (eget kapitel) Krav F2 (eget kapitel) Krav F3 (eget kapitel) Krav F4 (eget kapitel) Referenser Startrapporten ska skickas till chefen före projektstart för godkännande. Leveransen ska ske till plattformen Itslearning. Namnet på filen ska vara ”fornamn_efternamn_HWSW_engjob_a.zip”.	Ja
Förstudien slut (ska godkännas före nästa steg) Start med slut leveransen		
Konstruktionskod mm		
001	Regler och riktlinjer för VHDL och C ska följas	Ja
Dokumentationskrav och struktur på mapparna		
002	Leverans med följande rapport: 1. Rapport (**OBS** filnamn: fornamn_efternamn_HWSW_eng_B) a. Första sida med beskrivande titel (INTE Ingenjörsjobb), namn, datum, e-postadress och kort sammanfattning b. Innehållsförteckning c. Kravspecifikation (eget kapitel) d. Äldre projekt (eget kapitel) e. Tidplan, uppskattat antal timmar och verkligt antal timmar. Kostnad för projektet (eget kapitel) f. Beskriv hur ”bootningen” går till (eget kapitel) g. Konstruktionsbeskrivning HW (översikt arkitektur, varje block o.s.v. beskriv hur metastabilitets problem undviks, flödesschema eller liknande på redan utvecklade IP-komponenter, storlek, uppskattad effektförbrukning och inställningar på verktygen) (eget kapitel med underkapitel) h. Konstruktionsbeskrivning SW (översikt arkitektur, ev. flödesschema, storlek, inställningar på verktygen o.s.v.) (eget kapitel med underkapitel) i. Verifiering av HW komponenterna (testbänkar med förklaring till HW) (eget kapitel). Ingen verifiering av hela systemet, bara subsystem j. Validering, visas med en film (länk), se nästa krav (eget kapitel) k. Analys, storlek och resultat av eventuella optimeringar (eget kapitel) l. Slutsatser, reflektioner och framtida förbättringar (eget kapitel) m. Referenser n. Bilagor o. Leverans i formatet Word eller pdf, svenska eller engelska. OBS! var konsekvent och välj ett språk.	Ja

003	Valideringsbevis och presentation av resultat; Skapa en film (lägg gärna upp den på Youtube), ta gärna med en introduktion med HW och SW arkitekturen, visa tydligt på hur systemet ”bootar” efter spänningspåslag och visa hur det fungerar. En rekommendation är att lägga till ditt namn (frivilligt), bra reklam. Trevligt om taggen ”TEIS” läggs till, då blir det enkelt att få fram alla filmerna från TEIS utbildningen. **OBS, Om du inte vill att den ska publiceras, skriv det tydligt efter film länken.**	
Leveranskrav		
004	Slutleveransen ska innehålla två mappar med följande dokument: • **Mappen Dokumentation** o Konstruktionsrapporten (se rapportkrav) o Eventuellt fristående bilagor som kod, kretsschema och datablad • **Mappen Konstruktionsunderlag** o Arkiverat Quartus projekt, o Software mappen o IP komponenter i en map • **Mappen Konfigureringsfiler** o configuration.sof, pof • **Mappen diverse** Leveransen ska ske till plattformen Itslearning, Engineering job. Namnet på filen ska vara ”fornamn_efternamn_HWSW_eng.zip” (en fil).	Ja

3 Beskrivning av två gamla projekt.

Nedan beskrivs först ett projekt av Lars Bengtsson från 2015 och sedan ett projekt av André Norberg från 2015.

3.1 Lars Bengtsson: Space Invaders

Ett antal ”monster” rör sig på ”himlen” i horisontell led och spelarens uppgift är att skjuta ner dem.

Spelet baseras på en extern grafisk display som ansluts till DE2-115-kortet GPIO-kontakt. En Qsys-system för att lägga på IP-komponenterna som kommer att använde i projektet(onchip-ram, sram, sierra, display_hw_ip, etc..). Spelet styrs med fyra knappar på DE2-115-kortet. Konstruktionslösning bygger till stor del på multitasking med hjälp av realtidskärnan Sierra.

Hela systemet i en .pof-fil (HW+SW).

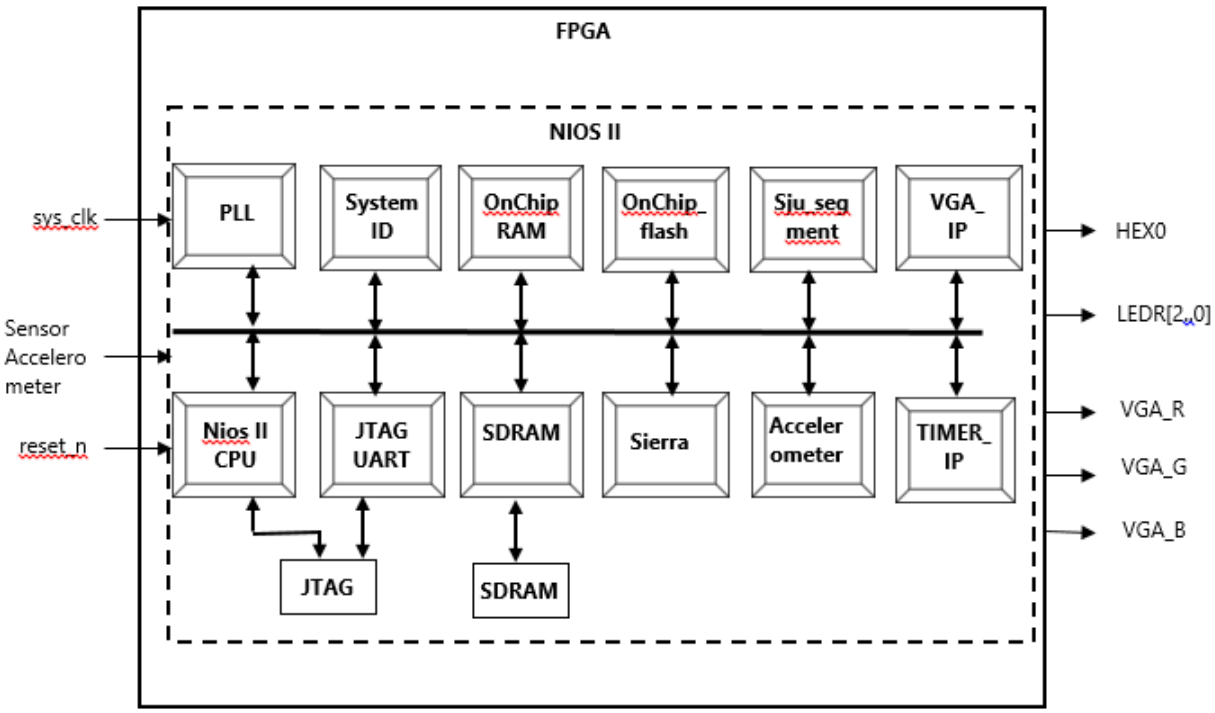
3.2 André Norberg: Rabbit vs Robots 2

Denna är ett spel projektet som är konstruerar för kan körs med en accelerometer eller med fyra knappar, lysdioder för att visar vilken riktning, och vilken hastighet spelaren förflyttar sig i. Sjusegmentdisplay för att för att visar några resultaten från spelet

Spelet baseras på en DE2-115-kortet. En Qsys-system för att lägga på IP-komponenterna som kommer att använde i projektet(onchip-memory, sierra, vga_controller_0, green_led, red_led, i2c_interface, etc..). Konstruktionslösning bygger till stor del på multitasking med hjälp av realtidskärnan Sierra med fem tasking (Idle, Gameflow, Robotmove, Task_IRQ, LCD_LED).

Hela systemet i en .pof-fil (HW+SW).

4 Första arkitektur



Figur 1. HW/SW Systemarkitektur

4.1 Pll

En PLL finns tillgänglig i systemet för att distribuera varianter av systemklockan på 50 MHz till delar av systemet.

4.2 System ID

En komponent som innehåller information om hårdvaruversion.

4.3 OnChip_Ram

Intern styrenhet för SDRAM-minnet.

4.4 OnChip_flash

I detta minne kan information sparas även när spänningen är frånslagen. Den process bootning är till nästa version.

4.5 Sju_segment ip

Styr för 7-segment display utgången på FPGA kor för att visar vilken task exekverar.

4.6 Vga ip

Skapar Avalon bus interface av läsning/skrivning även instansierar komponenten som skickar utgången signaler till skärmen.

4.7 Jtag uart

Kommunikationsgränssnitt mellan USB Blaster och data/instruktionsbus. Används för kommunikation mellan PC och CPU för programmering och debuggning.

4.8 Sdram

Extern styrenhet för SDRAM-minnet.

4.9 Sierra ip

Sierra Ral-Time Kernel(RTK-accelerator) för att exekverar tasking.

4.10 Accelerometer spi

Skapa kommunikation interface för att läsa koordinaterna från DE10-Lite kortet.

4.11 Timer ip

Filen som skapar Avalon bus interface av läsning/skrivning även instansierar komponent som använder för att ändrar hastighet frekvensen för data visualisering på skärmen.

5 PROJEKPLAN

Aktivitets - och tidplan för ingenjörsjobbet kan ses i Tabell 2. Totalt uppskattas ingenjörsjobbet ta 28 timmar/800kr(timme), vilket ger en lönekostnad på 16 800 SEK.

Tabell 2. Aktivitets- och tidsplan för ingenjörsjobbet

Aktivitet	Veckor				
	36	37	38	Timmar(h)	Kostnad(kr)
Kravspec./förstudie	5			5	4 000
Testprotokoll/ Drivrutiner	1	1	1	3	2 400
Arkitekturbeskrivning	3	3		6	4 800
Programmering av visuellt gränssnitt	4	3	1	7	5 600
Verifiering		1	1	2	1 600
Validering			1	1	8 00
Slutrapport		2	2	4	3 200
Summa tid	13	10	6	28	22 400

6 Bootningen beskrivning.

DE10-Lite kortet innehålla en ett internt flashminne som kan sparas information även när kortet får inte ingen spänning. Det interna flashminne är uppdelat i två delar som är CFM (Configuration Flash Memory) var hårdvarukonfiguration (.sof) sparas och UFM (User Flash Memory) var mjukvara sparas (.hex).

Vid uppstart kan både hårdvara och mjukvara laddas in i FPGA:n. Först kommer hårdvaran att laddas upp, och sedan används Nios CPU:n att flytta mjukvaran från UFM till det minne som ska användas för att exekvera programkoden. Denna kopieringsprocess kallas för bootloader.

Efter konverter med verktyget "Concert program files", hårdvaran och mjukvaran läggs i en fil som heter .pof. Denna fil kommer sedan att bli uppdelad i två olika minnen inuti FPGA:n.

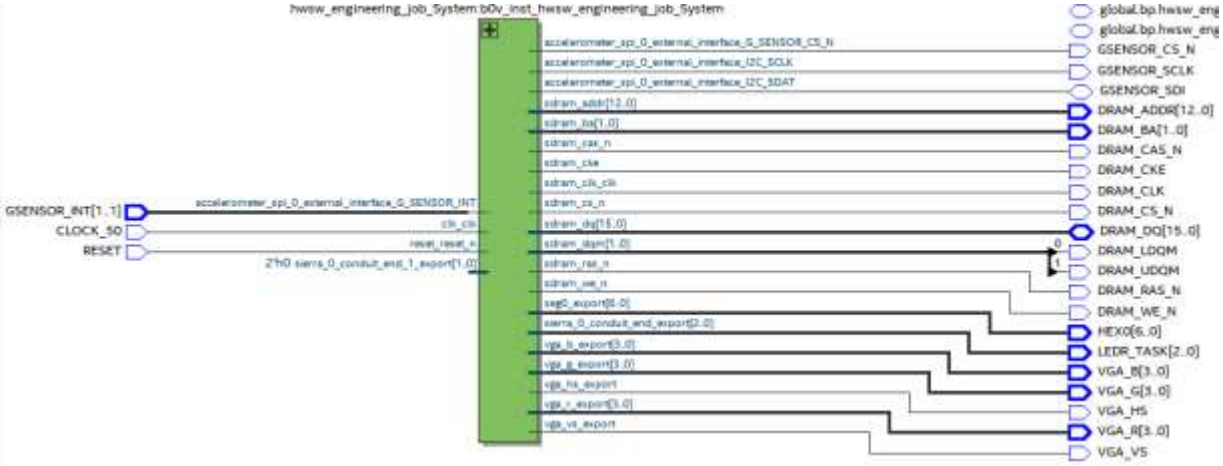
Det går att välja om Niosll CPU:n ska skicka programmet till ett internt minne (Dynamic Memory inuti kretsen), eller ett externt SDRAM minne (Dynamic Memory utanför kretsen).

7 Hårdvaran konstruktions beskrivning

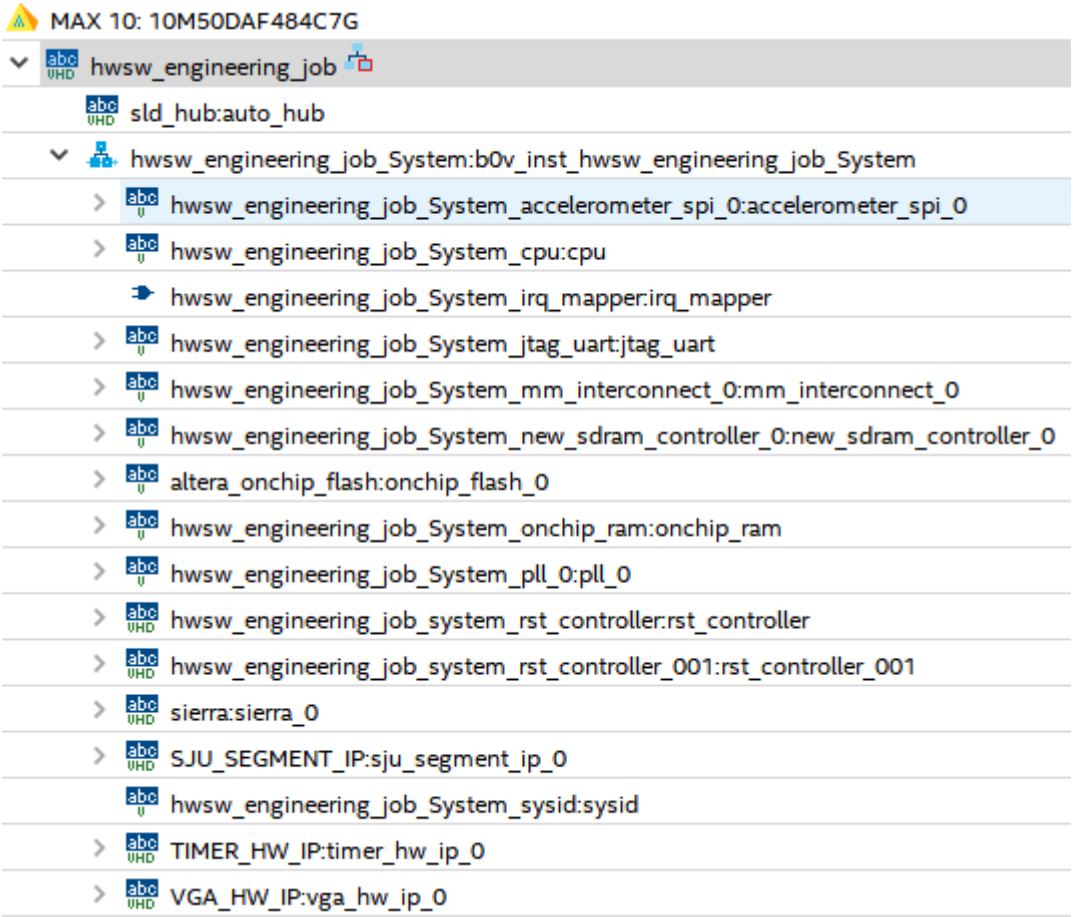
Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en accelerometer som skicka koordinater som kan kontrollera efter behov, en sjusegmentdisplay för att visa task exekveringen och en drivrutiner för att kontrollerar visualiseringen på vga skärmen.

7.1 Systemarkitektur

TOPPNIVÅN arkitektur visas i Figur 2, i figur 3 visar komponenthierarki och i figur 4 visar QSYS interface.



Figur 2. Systemarkitekturen



Figur 3. Komponenthierarki



Figur 4. QSYS interface

7.2 Delsystem

Systemet innehålla följande komponenter:

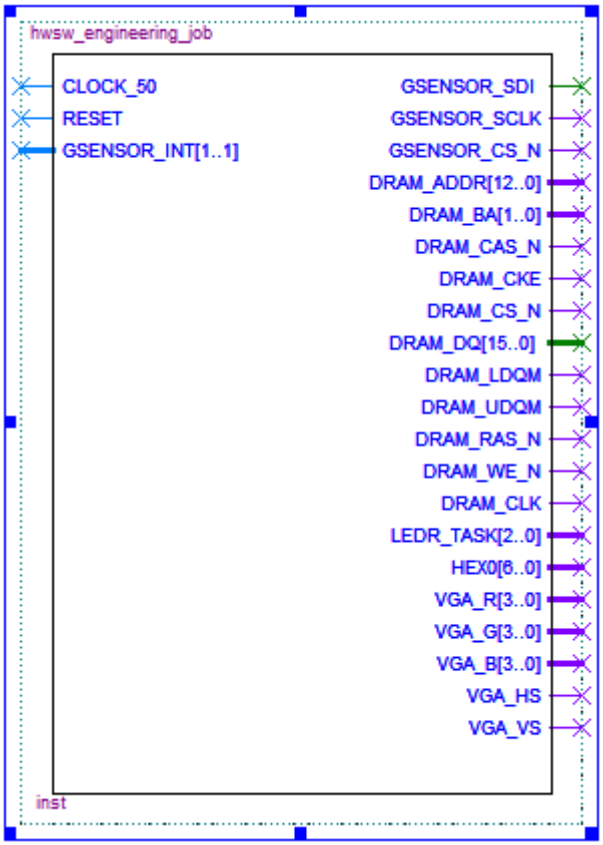
7.2.1 Toppnivån

Komponent namn: ***HWSW_ENGINEERING_JOB***

7.2.1.1 Systemarkitektur

Den toppnivån användas för att instansierar Qsys interface son innehålla hela IP komponenterna. Toppnivån får insignaler från accelerometer och skickar utsignaler till VGA skärmen och sjusegmentdisplay.

TOPPNIVÅN symbol visas i Figur 5 och i Tabell 3 visar In- och utsignalerna.



Figur 5. TOPPNIVÅN system

Tabell 3. In- och utsignalerna till TOPP filen

Signal	**Namn**	**Riktning**	**Typ**
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset (SW 9)	RESET	in	std_logic
Interrupt pin	GSENSOR_INT	in	std_logic_vector(1 downto 1)
I2C serial data	GSENSOR_SDI	out	std_logic
I2C serial clock	GSENSOR_SCLK	out	std_logic
Chip Select	GSENSOR_CS_n	out	std_logic
Läs tasking	LEDR_TASK	out	std_logic_vector(2 downto 0)
Sjusegment	HEX0	out	std_logic_vector(6 downto 0)
Horizontell synkronism	vga_hs	out	std_logic
Vertikal synkronism	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)
External minne	SDRAM	out	std_logic, std_logic_vector

Ingången signaler

CLOCK_50: 50Mhz input clock.
RESET (reset_n): Reset av signalen. Active low.
gsensor_int: Interrupt pin 1.

Utgången signaler

LEDR_TASK: Visar task tiden som tar det att exekverar i processor.
HEX0: Sjusegmentdisplay för att visar vilken task är på exekvering.
SDRAM: Externa minne som innehålla data och instruktioner.
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram(framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

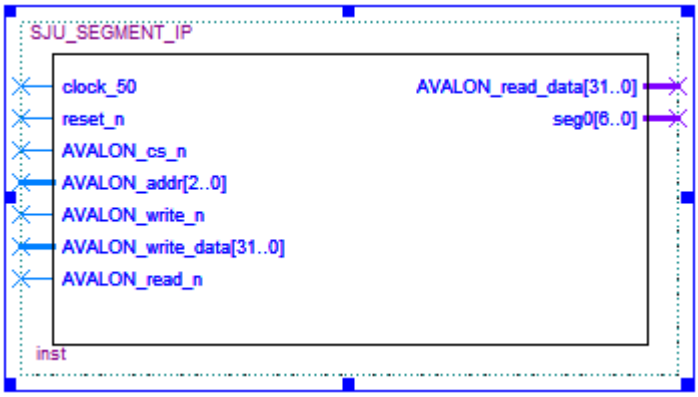
7.2.2 Sjusegmentdisplay IP - komponents

Komponent namn: ***SJU_SEGMENT_IP***

7.2.2.1 Systemarkitektur

När sierra-accelerator exekverar tasking på Nios II skicka den utgången signal till avalon bussinterface för att visar genom den här komponenten vilken task exekverar sig.

SJU_SEGMENT_IP-komponents symbol visas i Figur 6 och i Tabell 4 visar In- och utsignalerna.



Figur 6. Komponenten för sjusegmentdisplay

Tabell 4. In- och utsignalerna till SJU_SEGMENT_IP filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset (SW 9)	reset_n	in	std_logic
Interrupt pin	AVALON_cs_n	in	std_logic
Adress bus in	AVALON_addr	in	std_logic_vector(2 downto 0)
Write enable	AVALON_write_n	in	std_logic
Data bus in	AVALON_write_data	in	std_logic_vector(31 downto 0)
Read enable	AVALON_read_n	in	std_logic
Data bus out	AVALON_read_data	out	std_logic_vector(31 downto 0)
Sjusegment	SEG0	out	std_logic_vector(6 downto 0)

Ingången signaler

CLOCK_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
avalon_cs_nt: Active low.
avalon_addr: skriver eller läser på komponenten.
avalon_write_n: aktiverar skrivning på Avalon interface.
avalon_read_n: aktiverar läsning på Avalon interface
avalon_write_data: Data bus ingång som skriver till komponent.

Utgången signaler

avalon_read_data: Data bus utgång som läser från komponent.
seg0: Läs utgången signal för att visar en värde på sjusegmentdisplay

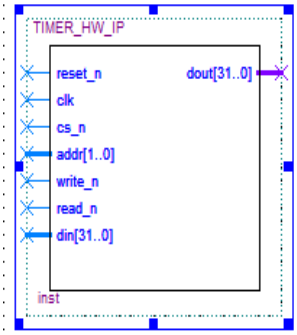
7.2.3 Timer IP - komponent

Komponent namn: ***TIMER_HW_IP - komponent***

7.2.3.1 Systemarkitektur

Den komponenten genererar en delay i processorn.

TIMER_HW_IP-komponents symbol visas i Figur 7 och i Tabell 5 visar In- och utsignalerna.



Figur 7. TIMER_HW_IP komponent

Tabell 5. In- och utsignalerna till TIMER_HW_IP filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	RESET_N	in	std_logic
Chip select	CS_N	in	std_logic
Address	ADDR	in	std_logic_vector(1 downto 0)
Write Enable	WRITE_N	in	std_logic
Read Enable	READ_N	in	std_logic, std_logic_vector
Write data 32 bitar	DIN	in	std_logic_vector(31 downto 0)
Read data 32 bitar	DOUT	out	std_logic_vector(31 downto 0)

Ingången signaler

CLOCK_50: 50Mhz input clock.
sw(reset_n): Reset av signalen. Active low.
cs_n: Det blir noll för aktivering läsning eller skrivning.
address: Två bitar. Offset noll till address=”00” och offset fira till address=”01”.
write_n: aktiverar med ”0” för skrivning på Avalon interface
read_n: aktiverar med ”0” för läsning på Avalon interface
din: Data bus ingång som skriver till komponenten

Utgången signaler
dout: Data bus utgång som läser från komponenten

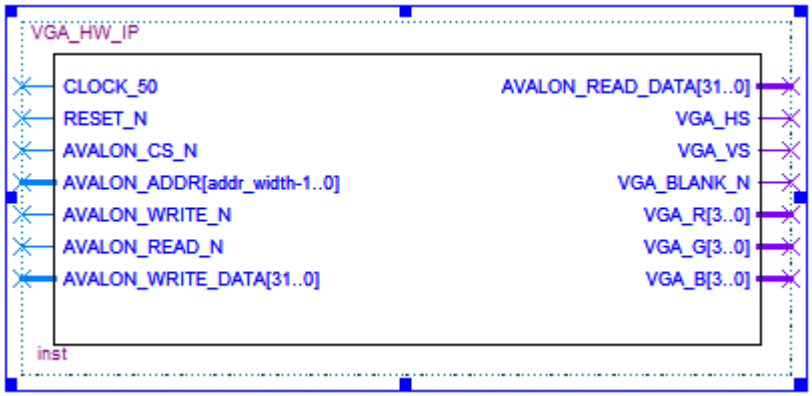
7.2.4 Vga IP - komponent

Komponent namn: ***VGA_HW_IP***

7.2.4.1 Systemarkitektur

CPU'n genom denna ip_komponent läs eller skriv i busen Avalon för att skickar information till skärmen.

VGA_HW_IP-komponents symbol visas i Figur 8 och i Tabell 6 visar In- och utsignalerna.



Figur 8. VGA_HW_IP komponent

Tabell 6. In- och utsignalerna till TIMER_HW_IP filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	RESET_N	in	std_logic
Chipselect	AVALON_CS_N	in	std_logic
Adress bus in	AVALON_ADDR	In	std_logic_vector(16 downto 0)
Write enable	AVALON_WRITE_N	In	std_logic
Read enable	AVALON_READ_N	In	std_logic
Data bus in	AVALON_WRITE_DATA	In	std_logic_vector(31 downto 0)
Data bus out	AVALON_READ_DATA	out	std_logic_vector(31 downto 0)
Horizontell synk	VGA_HS	out	std_logic
vertikal synk	VGA_HS	out	std_logic
Röd signal	VGA_R	out	std_logic_vector(3 downto 0)
Grön signal	VGA_G	out	std_logic_vector(3 downto 0)
Blå signal	VGA_B	out	std_logic_vector(3 downto 0)

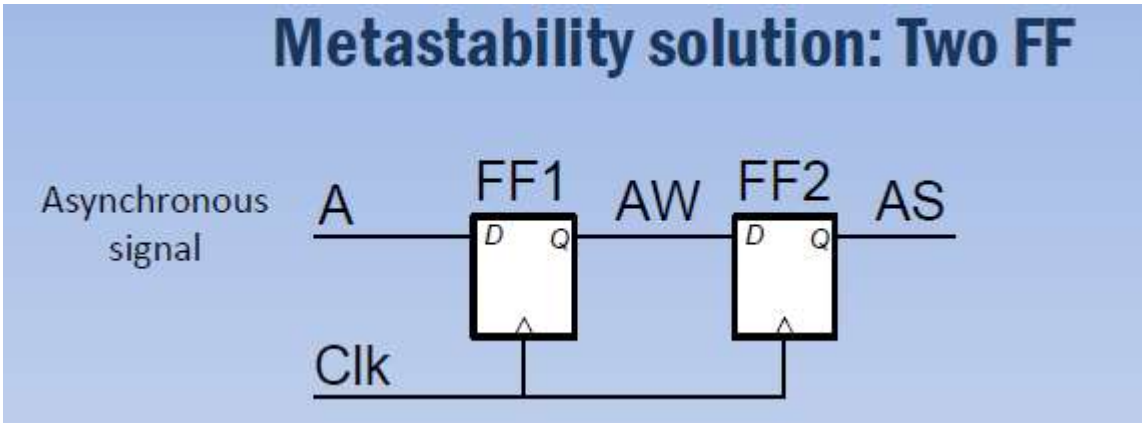
Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.

avalon_cs_nt: Active low.
avalon_addr: skriver eller läser på komponenten.
avalon_write_n: aktiverar skrivning på Avalon interface.
avalon_read_n: aktiverar läsning på Avalon interface
avalon_write_data: Data bus ingång som skriver till komponent.

Utgången signaler
avalon_read_data: Data bus utgång som läser från komponent.
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram (framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

7.2.5 Metastabilitet lösning

Att använda snabbare flipflops minskar setup och hold tiderna för flipflops , den här minskar tiden som flipflops är sårbar för metastabilitet. När ingångsfrekvensen för en asynkron signal sänks, minskas också chansen för att ingången ändras under setup och hold.
Nästa figur visar som lägg in två vippor i rad för asynkron signaler.



7.3 I/O pinar

Figur 9 visar pinar för DE10-Lite kortet.

Node Name	Direction	Location
CLOCK_50	Input	PIN_N14
DRAM_ADDR[12]	Output	PIN_R20
DRAM_ADDR[11]	Output	PIN_P20
DRAM_ADDR[10]	Output	PIN_T20
DRAM_ADDR[9]	Output	PIN_P19
DRAM_ADDR[8]	Output	PIN_P18
DRAM_ADDR[7]	Output	PIN_R18
DRAM_ADDR[6]	Output	PIN_T19
DRAM_ADDR[5]	Output	PIN_T18
DRAM_ADDR[4]	Output	PIN_U19
DRAM_ADDR[3]	Output	PIN_U18
DRAM_ADDR[2]	Output	PIN_V18
DRAM_ADDR[1]	Output	PIN_W19
DRAM_ADDR[0]	Output	PIN_U17
DRAM_BA[1]	Output	PIN_T22
DRAM_BA[0]	Output	PIN_T21
DRAM_CAS_N	Output	PIN_U21
DRAM_CKE	Output	PIN_N22
DRAM_CLK	Output	PIN_L14
DRAM_CS_N	Output	PIN_U20
DRAM_DQ[15]	Bidir	PIN_F22
DRAM_DQ[14]	Bidir	PIN_G19
DRAM_DQ[13]	Bidir	PIN_G20
DRAM_DQ[12]	Bidir	PIN_G22
DRAM_DQ[11]	Bidir	PIN_H22
DRAM_DQ[10]	Bidir	PIN_H21
DRAM_DQ[9]	Bidir	PIN_J22
DRAM_DQ[8]	Bidir	PIN_P21
DRAM_DQ[7]	Bidir	PIN_V21
DRAM_DQ[6]	Bidir	PIN_W20

Node Name	Direction	Location
DRAM_DQ[5]	Bidir	PIN_W22
DRAM_DQ[4]	Bidir	PIN_Y22
DRAM_DQ[3]	Bidir	PIN_AA21
DRAM_DQ[2]	Bidir	PIN_AA22
DRAM_DQ[1]	Bidir	PIN_Y20
DRAM_DQ[0]	Bidir	PIN_Y21
DRAM_LDQM	Output	PIN_V22
DRAM_RAS_N	Output	PIN_U22
DRAM_UDQM	Output	PIN_J21
DRAM_WE_N	Output	PIN_V20
GSENSOR_CS_N	Output	PIN_AB16
GSENSOR_INT[1]	Input	PIN_Y14
GSENSOR_SCLK	Output	PIN_AB15
GSENSOR_SDI	Bidir	PIN_V11
HEX0[6]	Output	PIN_C17
HEX0[5]	Output	PIN_D17
HEX0[4]	Output	PIN_E16
HEX0[3]	Output	PIN_C16
HEX0[2]	Output	PIN_C15
HEX0[1]	Output	PIN_E15
HEX0[0]	Output	PIN_C14
LEDR_TASK[2]	Output	PIN_A10
LEDR_TASK[1]	Output	PIN_A9
LEDR_TASK[0]	Output	PIN_A8
RESET	Input	PIN_F15
VGA_B[3]	Output	PIN_N2
VGA_B[2]	Output	PIN_P4
VGA_B[1]	Output	PIN_T1
VGA_B[0]	Output	PIN_P1
VGA_G[3]	Output	PIN_R1

Node Name	Direction	Location
VGA_B[2]	Output	PIN_P4
VGA_B[1]	Output	PIN_T1
VGA_B[0]	Output	PIN_P1
VGA_G[3]	Output	PIN_R1
VGA_G[2]	Output	PIN_R2
VGA_G[1]	Output	PIN_T2
VGA_G[0]	Output	PIN_W1
VGA_HS	Output	PIN_N3
VGA_R[3]	Output	PIN_Y1
VGA_R[2]	Output	PIN_Y2
VGA_R[1]	Output	PIN_V1
VGA_R[0]	Output	PIN_AA1
VGA_VS	Output	PIN_N1

Figur 9. Pinnar för DE10-Lite kortet

7.4 Frekvens, storlek, effektförbrukning uppskattning

Denna del visar max frekvens för hela konstruktionen och varje delsystem.

7.4.1 Max frekvens för hela konstruktionen

Denna konstruktion är kompilerade i en 50 MHZ som systemklocka och har det en klocka av 25 MHZ(CLK_25) för att köras med VGA skärmen. Resurser som används i konstruktionen det passade bra på VGA-kortet men kan det optimerar sig bättre med advisor verktygen.

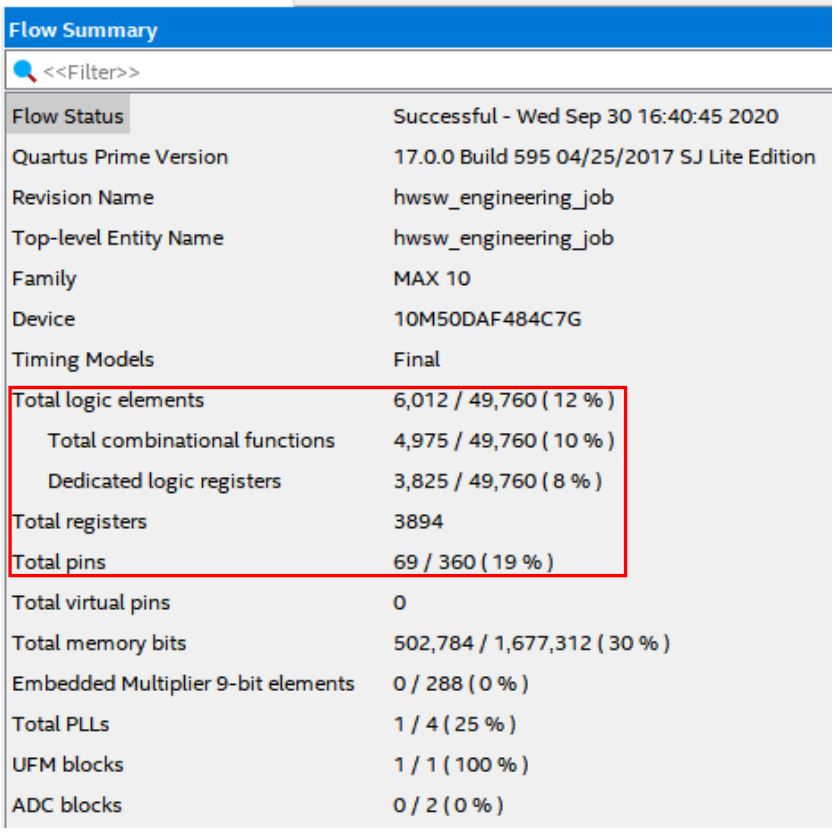
Tabell 7 visar max frekvens för hela konstruktionen.

Tabell 7: Max frekvens av konstruktionen.

Modell	**Fmax systemklocka**
Slow 1200mV 85C Model	79.23 MHz
Slow 1200mV 0C Model	85.35 MHz
Modell	**Fmax CLK_25**
Slow 1200mV 85C Model	190.69 MHz
Slow 1200mV 0C Model	209.82 MHz

7.4.2 Storlek för hela konstruktionen och del system

Figur 10 visar konstruktionens resursanvändning, figur 10 visar delsystem resursanvändning.



Figur 20: Konstruktionens resursanvändning.

Figur 10 visar resursanvändning för varje delsystem. Nummer som är utanför parentes indikerar antal av resurser som kan använda varje entity och nummer som är innanför parentes indikerar antal av resurser som används varje entity.

Analysis & Synthesis Resource Utilization by Entity

<<Filter>>

	Compilation Hierarchy Node	Combinational ALUTs	Dedicated Logic Registers	Memory Bits	UFM Blocks	DSP Elements	DSP 9x9	DSP 18x18	Pins	Virtual Pins	ADC blocks
1	|hwsw_engineering_job	4959 (1)	3860 (0)	502784	1		0	0	69	0	0
1	|hwsw_engineering_job:gineering_job_System|	4808 (0)	3775 (0)	502784	1		0	0	0	0	0
1	|SJU_SEGMENT_IP:sju_segment_ip_0|	59 (37)	64 (64)	0	0		0	0	0	0	0
2	|TIMER_HW_IP:timer_hw_ip_0|	65 (1)	34 (2)	0	0		0	0	0	0	0
3	|VGA_HW_IP:vga_hw_ip_0|	164 (2)	37 (4)	230400	0		0	0	0	0	0
4	|altera_onchip_flash:onchip_flash_0|	535 (0)	278 (0)	0	1		0	0	0	0	0
5	|hwsw_engineering_jo..accelerometer_spi_0|	136 (45)	109 (37)	0	0		0	0	0	0	0
6	|hwsw_engineering_job_System_cpu:cpu|	1028 (0)	593 (0)	10240	0		0	0	0	0	0
7	|hwsw_engineering_job..._jtag_uart:jtag_uart|	295 (34)	359 (13)	0	0		0	0	0	0	0
8	|hwsw_engineering_j...mm_interconnect_0|	1191 (0)	1260 (0)	0	0		0	0	0	0	0
9	|hwsw_engineering_jo..._sdram_controller_0|	271 (215)	248 (156)	0	0		0	0	0	0	0
10	|hwsw_engineering_j...hip_ram:onchip_ram|	1 (1)	0 (0)	262144	0		0	0	0	0	0
11	|hwsw_engineering_job_System_pll_0:pll_0|	8 (7)	6 (2)	0	0		0	0	0	0	0
12	|hwsw_engineering_job_...rollerrst_controller|	6 (0)	16 (0)	0	0		0	0	0	0	0
13	|hwsw_engineering_job...1:rst_controller_001|	1 (0)	3 (0)	0	0		0	0	0	0	0
14	|sierra:sierra_0|	1048 (0)	768 (0)	0	0		0	0	0	0	0
2	|sld_hubrauto_hub|	150 (1)	85 (0)	0	0		0	0	0	0	0

Figur 10: Delsystem resursanvändning.

7.4.3 Effektförbrukning uppskattning

Figur 11 visar konstruktionens effektförbrukning uppskattning. Den totala förbrukningen beräknades till 152,02 mW.

PowerPlay Power Analyzer Summary

<<Filter>>

PowerPlay Power Analyzer Status	Successful - Wed Sep 30 20:00:16 2020
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	hwsw_engineering_job
Top-level Entity Name	hwsw_engineering_job
Family	MAX 10
Device	10M50DAF484C7G
Power Models	Final
Total Thermal Power Dissipation	152.02 mW
Core Dynamic Thermal Power Dissipation	0.61 mW
Core Static Thermal Power Dissipation	96.39 mW
I/O Thermal Power Dissipation	55.02 mW

Figur 11: Effektförbrukning med toggle rate på 12,5% och typiska driftförhållanden.

8 Mjukvaran konstruktions beskrivning

Till denna konstruktion har en Nios2 processor använts, och har programmerats i C med utvecklingsverktyget Nios2 – Eclipse.

8.1 Minneskarta

Memory-map från ”sumary.html” för konstruktionen, se figur 12.

Nios II Memory Map

Slave Descriptor	Address Range	Size	Attributes
accelerometer_spi_0	0x08491458 - 0x08491459	2	
jtag_uart	0x08491450 - 0x08491457	8	printable
sysid	0x08491448 - 0x0849144F	8	
onchip_flash_0_csr	0x08491440 - 0x08491447	8	
TIMER_HW_IP_0	0x08491430 - 0x0849143F	16	
pll_0	0x08491420 - 0x0849142F	16	
SJU_SEGMENT_IP_0	0x08491400 - 0x0849141F	32	
sierra_0	0x08491000 - 0x084913FF	1024	
onchip_ram	0x08488000 - 0x0848FFFF	32768	memory
VGA_HW_IP_0	0x08400000 - 0x0847FFFF	524288	
onchip_flash_0_data	0x08200000 - 0x0835FFFF	1441792	flash, memory, non-volatile
new_sdram_controller_0	0x04000000 - 0x07FFFFFF	67108864	memory

Figur 12. Flödesdiagram

8.2 Task

Konstruktionen använder multitaskinglösning Sierra RTK där 5 task är tillagda. Dessa är:

- IDLE
- Accelerometer_Task
- Sjusegment_Task
- Timer_Task_1
- Timer_Task_2

8.2.1 Idle

Den task kommer det att fungerar när systemet inte gör något. Prioritet = 0 under task exekveringen.

8.2.2 Accelerometer_Task

Läser koordinatorer(x,y,z) från accelerometer och skriva ut det på Nios II konsol. Prioritet = 2 under task exekveringen.

8.2.3 Sjusegment_Task

Den task använder semaphore för att synkronisering och till ”mutual exclusion” av task. När semaphore är ledigt kommer det att skriva ut nummer task prioritet som är på exekvering på sjusegmentdisplay på kortet och X-koordinat på VGA-skärmen. Prioritet = 1 under task exekveringen.
Dessutom kommer det att skriva ut på VGA-skärmen accelerometer koordinaterna beroende på prioritetsordning.

8.2.4 Timer_Task_1

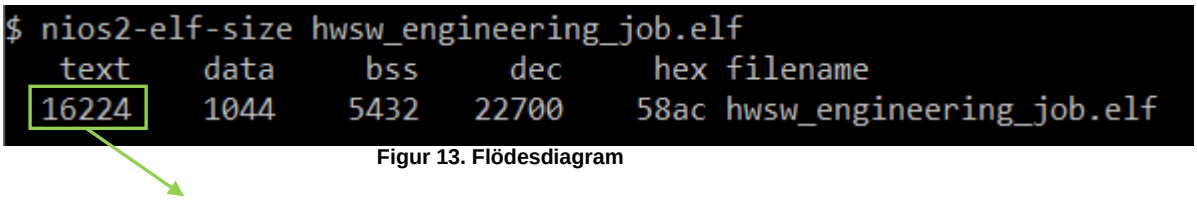
Spara det värde på en ”global variabel” för att skriva ut Y-koordinat på VGA-skärmen. Prioritet = 3 under task exekveringen.

8.2.5 Timer_Task_2

Spara det värde på en ”global variabel” för att skriva ut Z-koordinat på VGA-skärmen. Prioritet = 4 under task exekveringen.

8.3 Storlek

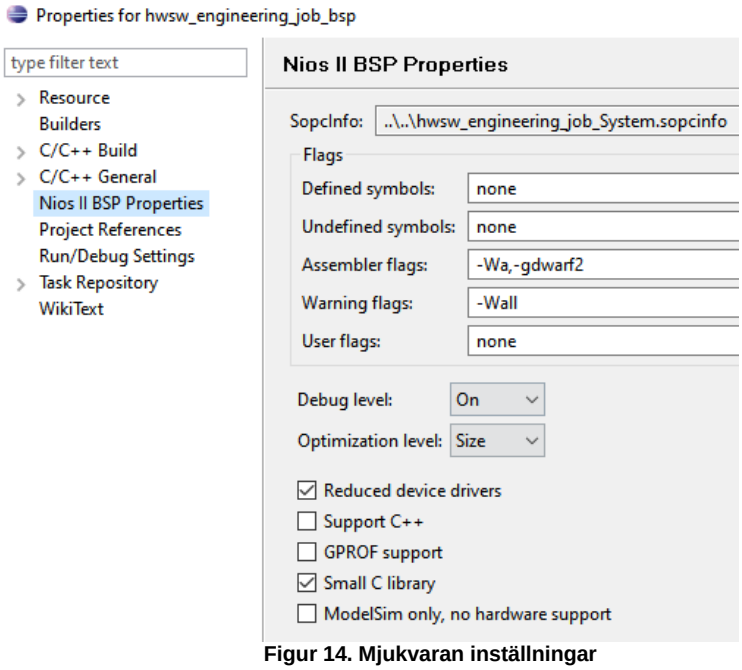
I FIGUR 13 visar en total storlek av 22 700 KB i hela system efter optimeringen.



Figur 13. Flödesdiagram

8.4 Inställningar

Enligt manualen för Sierra RTK gick det att aktivera ”Reduced device drivers” och ”Small C library” vilket är gjort i detta projekt. Se FIGUR 14.



Figur 14. Mjukvaran inställningar

9 Validering

En film finns utlagd på Youtube. Länken är:

https://www.youtube.com/watch?v=RfFZ5gjMoFQ&ab_channel=Darwinrr

10 Analys

Igenom ”Resource Optimization Advisor” slå på optimering för area i kompilatorn, ange i syntetiseraren att använda minimalt med resurser och aktivera mappning av logik till minne.
Igenom ”Timing Optimization Advisor” kommer att ha en högre maxfrekvens kompilera genom optimering för klockhastighet, att sätta på ”router timing optimization level” som maximum. Alla dessa rekommendationer använder mer resurser för att ha en bättre I/O timing och för att ha en bättre "Hold Time & Minimum Delay Timing".
Igenom ”Power Optimization Advisor”, den konstruktionen kommer att ha en bättre optimering för att at en effektiv effektförbrukning. Den kommer att försöka maximera prestanda, minimera användningen av logik eller balansera hög prestanda med minimal användning av logik. Vissa av dessa tips ger ökad resursanvändning och andra lägre prestanda.

11 Slutsatser

I den här versionen kunde jag inte inkludera en watchdog timer eftersom hade jag problem när jag ville jobbar med ”booning”, det fanns problem när jag inkluderar watchdog timer på sierra-multitaskin på C-program, efter gjorde jag hela bootning process testade jag booning och hända det inget, det ser ut som exekverar hårdvaran bara.
En framtida förbättringar är analyserar problemet för att inkluderar mjukvaran-watchdog timer för att systemet startas om automatiskt om den hänger sig